

MAP COLORING PROBLEM FOR TELANGANA DISTRICTS USING CSP

1. Title

Implementation of Map Coloring Problem for Telangana Districts using Constraint Satisfaction and Backtracking

2. Introduction

Map coloring is a well-known problem in Artificial Intelligence and is a simple example of a **Constraint Satisfaction Problem (CSP)**.

The basic idea is to color different regions on a map in such a way that two regions sharing a common boundary should not have the same color.

In this experiment, the districts of **Telangana** are considered as the regions of the map. Each district is represented as a node in a graph, and a connection is made between two nodes when the corresponding districts are neighboring districts.

The objective is to assign colors to all the districts while making sure that no two neighboring districts receive the same color.

This problem is solved using a **backtracking algorithm**, which is a commonly used technique for solving constraint satisfaction problems.

3. Problem Statement

The Telangana district map needs to be colored using a limited number of colors.

The following conditions must be satisfied:

1. Every district must receive exactly one color.
2. Neighboring districts must have different colors.
3. A district can only use one of the available colors.
4. If a particular color assignment causes a conflict, the algorithm should go back and try another color.

For this experiment, four colors are considered:

- Red
- Green

- Blue
- Yellow

The goal is to find a valid coloring for all the districts.

4. Aim

The aim of this experiment is:

To implement the Map Coloring Problem for Telangana districts using a Constraint Satisfaction Problem and a backtracking algorithm, ensuring that no two neighboring districts have the same color.

5. Objectives

The main objectives are:

- To understand the Map Coloring Problem.
 - To represent the Telangana districts as a graph.
 - To identify neighboring districts.
 - To define colors as possible values for each district.
 - To implement the CSP approach.
 - To use backtracking to find a valid solution.
 - To visualize the resulting district-color assignments.
 - To verify that there are no color conflicts between neighboring districts.
-

6. Concept of Map Coloring

Suppose we have three neighboring regions:



If we have three colors:

Red
Green
Blue

we can assign:

A = Red
B = Green
C = Blue

This is valid because every pair of neighboring regions has a different color.

The same principle is applied to the districts of Telangana.

The main constraint is:

$$Color(A) \neq Color(B)$$

whenever A and B are neighboring districts.

7. Constraint Satisfaction Problem

The Telangana map coloring problem can be represented as a **Constraint Satisfaction Problem (CSP)**.

A CSP consists mainly of three components:

7.1 Variables

The variables are the districts of Telangana.

For example:

- Hyderabad
- Rangareddy
- Medchal-Malkajgiri
- Sangareddy
- Medak
- Nizamabad
- Karimnagar
- Warangal
- Khammam
- Nalgonda
- and the remaining districts.

Each district represents one variable.

7.2 Domains

The domain represents the possible values that can be assigned to a variable.

Here, the possible values are the colors:

$$\text{Domain} = \{R, e, d, \text{GreenBlueYellow}\}$$

Therefore, every district can initially have any one of these four colors.

7.3 Constraints

The constraint is very simple:

Two neighboring districts cannot have the same color.

Mathematically:

$$X_i \neq X_j$$

for every pair of neighboring districts.

8. Graph Representation

The district map can be represented as a graph.

In the graph:

- **Vertices (nodes)** represent districts.
- **Edges** represent shared boundaries between neighboring districts.

For example:

```
Hyderabad ——— Rangareddy
      |
      |
Medchal-Malkajgiri
```

If Hyderabad and Rangareddy share a boundary, an edge is created between them.

Therefore:

Hyderabad → Rangareddy

means that these two districts cannot receive the same color.

9. District Adjacency Information

For the computer implementation, the district relationships are stored using a Python dictionary.

A simplified example is:

```
graph = {
    "Hyderabad": [
        "Medchal-Malkajgiri",
        "Rangareddy"
    ],

    "Medchal-Malkajgiri": [
        "Hyderabad",
        "Sangareddy",
        "Yadadri Bhuvanagiri"
    ],

    "Rangareddy": [
        "Hyderabad",
        "Sangareddy",
        "Vikarabad",
        "Mahabubnagar"
    ]
}
```

This means that Hyderabad has neighboring districts such as Medchal-Malkajgiri and Rangareddy.

The complete implementation contains the district adjacency information required by the experiment.

Note: For an academic submission, the adjacency list should be checked against the current official Telangana district boundary map because administrative boundaries and names can change.

10. Selection of Colors

Four colors are used:

Red
Green
Blue
Yellow

They are represented in Python as:

```
colors = [
    "Red",
    "Green",
```

```
"Blue",  
"Yellow"  
]
```

The algorithm tries these colors one by one for each district.

11. Why Backtracking is Used

Simply assigning colors randomly may create conflicts.

For example:

```
District A = Red  
District B = Green  
District C = Red
```

If A and C are neighboring districts, this assignment is invalid.

The algorithm therefore needs a method to undo an incorrect decision.

This is where **backtracking** is used.

Backtracking works like solving a puzzle:

1. Try a possible answer.
2. Check whether it is valid.
3. If it is valid, continue.
4. If it creates a problem, undo it.
5. Try another answer.

In simple terms:

Try → Check → Continue or Go Back → Try Again

12. Working of the Algorithm

The algorithm works as follows.

Step 1 – Select a district

The algorithm chooses a district that has not yet been colored.

Step 2 – Try a color

It starts with the first available color.

Step 3 – Check neighboring districts

The algorithm checks whether any neighboring district already has that color.

Step 4 – Assign the color

If there is no conflict, the color is assigned.

Step 5 – Move to the next district

The algorithm continues with another uncolored district.

Step 6 – Detect conflict

If none of the colors can be assigned without violating a constraint, the algorithm goes back to the previous district.

Step 7 – Change the previous color

The previous assignment is changed and the algorithm continues again.

Step 8 – Finish

The process continues until all districts have valid colors.

13. Example of Backtracking

Consider:

```
A — B
|   |
C — D
```

Suppose we first assign:

```
A = Red
B = Green
C = Green
```

Now suppose D is connected to both B and C.

D cannot be Green because B is Green.

If another constraint also prevents the remaining colors, the algorithm goes back and changes an earlier assignment.

This is called **backtracking**.

The same process is applied to the Telangana district graph.

14. Python Implementation

Step 1 – Import Libraries

```
import matplotlib.pyplot as plt
import networkx as nx
```

NetworkX is used to create and visualize the graph.

Matplotlib is used to display the graph.

Step 2 – Define Colors

```
colors = [
    "Red",
    "Green",
    "Blue",
    "Yellow"
]
```

Step 3 – Create Color Assignment Dictionary

```
color_assignment = {}
```

This dictionary stores the color assigned to each district.

For example:

```
Hyderabad → Red
Rangareddy → Green
Sangareddy → Blue
```

15. Checking Whether a Color is Safe

Before assigning a color, the program checks all neighboring districts.

```
def is_safe(district, color):
    for neighbour in graph[district]:
        if color_assignment.get(neighbour) == color:
```

```
        return False

    return True
```

The function returns:

True

if the color can safely be assigned.

Otherwise, it returns:

False

16. Backtracking Function

The main backtracking function is:

```
def map_coloring(index=0):

    if index == len(districts):
        return True

    district = districts[index]

    for color in colors:

        if is_safe(district, color):

            color_assignment[district] = color

            if map_coloring(index + 1):
                return True

            del color_assignment[district]

    return False
```

The important part is:

```
del color_assignment[district]
```

This removes the previous assignment when it causes a conflict.

That is the actual **backtracking step**.

17. Running the Algorithm

The algorithm is executed using:

```
if map_coloring():  
    print("Map coloring successful!")  
else:  
    print("No valid coloring found.")
```

If a valid assignment is found, the program displays:

```
Map coloring successful!
```

18. Displaying the Result

The district-color assignments can be displayed using:

```
for district, color in color_assignment.items():  
    print(  
        f"{district}: {color}"  
    )
```

The output will look similar to:

```
Hyderabad: Red  
Medchal-Malkajgiri: Green  
Rangareddy: Blue  
Sangareddy: Red  
Medak: Green  
Nizamabad: Blue  
...
```

The exact color assigned to each district can differ depending on the order of the districts and the colors.

That is acceptable as long as the constraints are satisfied.

19. Visualization

To make the result easier to understand, the districts can be represented as colored graph nodes.

```
G = nx.Graph()  
  
for district, neighbours in graph.items():  
    for neighbour in neighbours:  
        G.add_edge(  
            district,
```

```
        neighbour
    )
```

A position is then assigned to each node and the graph is displayed.

```
plt.figure(figsize=(16, 12))

pos = nx.spring_layout(
    G,
    seed=42
)

nx.draw(
    G,
    pos,
    with_labels=True,
    node_color=node_colors,
    node_size=1500,
    font_size=7
)

plt.title(
    "Telangana District Map Coloring"
)

plt.show()
```

The result visually shows which color has been assigned to each district.

20. Understanding the Visualization

The visualization uses colors such as:

```
Red    → One group
Green  → Second group
Blue   → Third group
Yellow → Fourth group
```

If two connected nodes have different colors, the constraint is satisfied.

For example:

Red — Green

is valid.

But:

Red — Red

is invalid because the two connected districts have the same color.

21. Validation of the Solution

After coloring all districts, the solution should be checked.

The important condition is:

$$\boxed{Color(D_i) \neq Color(D_j)}$$

for every neighboring pair.

We can use the following validation function:

```
def validate_coloring():  
    for district in graph:  
        for neighbour in graph[district]:  
            if color_assignment[district] == \  
               color_assignment[neighbour]:  
                return False  
    return True
```

Then:

```
if validate_coloring():  
    print(  
        "Valid coloring: No conflicts found."  
    )  
else:  
    print(  
        "Invalid coloring: Conflict found."  
    )
```

The expected result is:

```
Valid coloring: No conflicts found.
```

22. Measures of Effectiveness

The performance of the map-coloring algorithm can be measured using several factors.

22.1 Number of Colors Used

This indicates how many colors were required to color all districts.

$$ColorsUsed = | \{C_1, C_2, \dots, C_n\} |$$

22.2 Execution Time

This measures how long the program takes to find a valid solution.

```
import time

start_time = time.perf_counter()

map_coloring()

end_time = time.perf_counter()

execution_time = (
    end_time - start_time
)

print(
    "Execution Time:",
    execution_time,
    "seconds"
)
```

22.3 Number of Backtracks

The number of times the algorithm has to undo a previous decision can also be recorded.

A larger number of backtracks generally indicates that the search encountered more conflicts.

22.4 Constraint Violations

A valid final solution should have:

0 violations

22.5 Success

The final solution is successful if:

- Every district has a color.
 - No neighboring districts have the same color.
-

23. Expected Output

A successful experiment should produce:

```
Map coloring successful!
```

```
Valid coloring: No conflicts found.
```

followed by the color assigned to each district.

A graph visualization should also appear showing the colored nodes.

24. Advantages

The proposed method has several advantages:

1. It is simple to understand.
 2. It clearly demonstrates a CSP.
 3. It can find a valid coloring when one exists within the chosen color domain.
 4. It checks constraints at every assignment.
 5. It automatically corrects incorrect earlier decisions through backtracking.
 6. It can be applied to other geographical map-coloring problems.
 7. The graph representation makes the relationships between districts easy to understand.
-

25. Limitations

There are some limitations in this implementation.

1. Simplified Map

The graph representation does not reproduce the actual geographical shape of Telangana.

2. Adjacency Data

The solution depends on the correctness of the district-neighbor information supplied to the program.

3. Computational Cost

Basic backtracking can become expensive as the number of variables and constraints increases.

4. Administrative Changes

District boundaries and administrative information can change, so the adjacency data should be kept up to date.

26. Future Enhancements

The implementation can be improved using more advanced CSP techniques.

Minimum Remaining Values (MRV)

Instead of choosing districts in a fixed order, select the district with the fewest available colors.

Degree Heuristic

Choose the district having the largest number of neighboring districts.

Forward Checking

After assigning a color, remove that color from the possible choices of neighboring districts.

Real Geographical Visualization

Instead of showing a graph, a GIS-based Telangana district map can be used so that the actual district regions are filled with colors.

Comparison of Algorithms

The problem can also be solved using:

- Backtracking
- Forward Checking
- MRV
- Constraint Propagation

and their execution times and number of backtracks can be compared.

27. Applications

Map coloring has applications beyond geographical maps.

Examples include:

Timetable Scheduling

Different examinations or classes can be assigned time slots so that conflicting subjects do not receive the same slot.

Frequency Assignment

Nearby communication towers can be assigned different frequencies to avoid interference.

Register Allocation

Compilers can assign processor registers while avoiding conflicts.

Scheduling

Conflicting tasks can be assigned different time slots.

Geographic Information Systems

Different regions can be visually distinguished on maps.

28. Observation

The experiment shows that the Telangana district map can be represented as a graph and solved as a Constraint Satisfaction Problem.

The backtracking algorithm starts by assigning colors to districts and continuously checks whether the assignments satisfy the neighboring-district constraint.

Whenever a conflict occurs, the algorithm goes back and changes a previous assignment.

The final result is considered valid only when every district has been assigned a color and **no neighboring districts share the same color**.

29. Conclusion

The **Map Coloring Problem for Telangana districts** was successfully modeled as a Constraint Satisfaction Problem.

The districts were considered as variables, the available colors were considered as domains, and the neighboring relationships were considered as constraints.

A **backtracking algorithm** was implemented to systematically assign colors. Whenever an assignment caused a conflict, the algorithm reversed the previous decision and tried another color.

The experiment demonstrates an important Artificial Intelligence concept:

$$\boxed{\text{Variables} + \text{Domains} + \text{Constraints} = \text{CSP}}$$

The complete process can be summarized as:

